

데이터베이스 테이블 — 상세 가이드

업데이트: 2026-04-17 | 실제 운영 DB 기준 (5명 환자 처리 완료)

전체: 10개 테이블 | 브랜치: `feat/save-intermediate-results`

1. transcript_analysis_log (5행)

파이프라인 실행 기록 — "이 환자 파일이 처리되었다"는 사실과 처리 과정의 모든 메타데이터를 저장하는 테이블.

파이프라인이 `Input_Keystrokes_REC001 (SID 14).xlsx` 를 처리하면 이 테이블에 1행이 생깁니다. 처리 시작 시각 (`pipeline_started_at`), 분석된 문장 수 (`total_sentences=424`), 사용된 설정 (`top_n=10`), 결과 xlsx 백업 (`xlsx_data`), AI 파이프라인 완료 여부 (`processed=True`), 전체 상담 품질 점수 (`ai_overall_score=3.67`)가 기록됩니다. 다음에 같은 파일을 만나면 이 테이블을 확인하여 이미 처리된 파일은 건너뜁니다.

컬럼	타입	실제 값 (SID_10)	설명
<code>id</code>	SERIAL PK	1	분석 실행 고유 ID
<code>patient_id</code>	VARCHAR	SID_10	환자 식별자 (파일명에서 추출)
<code>total_sentences</code>	INT	428	R stringi 분리 후 문장 수
<code>top_n</code>	INT	10	도메인당 선택한 Top-K 수
<code>context_window</code>	INT	3	전후 ±3 문장 context 설정
<code>model_results</code>	JSONB	None	(폐기됨 — 이전 호환용으로 유지)
<code>xlsx_data</code>	BYTEA	(18,300 bytes)	결과 xlsx 백업. 디스크 파일 삭제 시 복구용
<code>source_filename</code>	VARCHAR	Input_Keystrokes_REC 001 (SID 10).xlsx	원본 입력 파일명
<code>pipeline_started_at</code>	TIMESTAMP	05:26:14	파이프라인 처리 시작 시각
<code>analyzed_at</code>	TIMESTAMP	05:26:30	NLP 결과 DB 저장 시각 (Step 8)
<code>ai_overall_score</code>	FLOAT	3.4	전체 도메인 ai_score 평균 (0-5). 의사 페이지에 표시
<code>processed</code>	BOOLEAN	True	전체 파이프라인 (NLP + AI) 완료 여부
<code>processed_at</code>	TIMESTAMP	05:29:28	AI 파이프라인 (Step 9) 완료 시각

사용처:

- `persistence.file_already_processed()` — 이미 처리된 파일 건너뛰기
- `GET /api/transcript/download/{patient_id}` — `xlsx_data` 에서 xlsx 다운로드 fallback
- `GET /api/transcript/history/{patient_id}` — 분석 실행 이력 조회
- `llm_domain_scoring_and_summary.analysis_id` FK 대상

2. sentence_prediction (250행 = 5환자 × 50문장)

NLP 모델 예측 결과 — 5개 NLP 분류 모델이 판단한 도메인별 Top-10 문장과 확률 점수를 저장하는 테이블.

예를 들어 SID 10의 428개 문장 중 cp(암 예후) 모델이 "so i'm going to take that 12 percent and cut it in half..."에 95.1% 확률을 매겨 해당 도메인 Top-10에 선택되었습니다. 5개 도메인 × 10개 문장 = 환자당 50행. 환자 페이지에서 "의사가 이런 말을 했습니다"라는 근거 문장을 보여줄 때 이 데이터를 사용합니다.

컬럼	타입	실제 값 (SID_10, cp)	설명
id	SERIAL PK	1	예측 고유 ID
analysis_id	INT FK	1	→ transcript_analysis_log.id
patient_id	VARCHAR	SID_10	환자 식별자
model	VARCHAR	cp	도메인 (cp/le/ed/inc/ius)
sentence_index	INT	166	전체 문장 순번
utterance_index	INT	67	원본 발화 번호 (i)
sentence_in_utterance	INT	3	발화 내 문장 번호 (i2)
speaker	VARCHAR	Interviewer:	화자
sentence_text	TEXT	"so i'm going to take that 12 percent and cut it in half..."	원본 문장
pred_score	FLOAT	0.951	NLP 확률 (95.1% cp 관련)
context	TEXT	"it removes the testosterone... <main>so i'm going to... </main>..."	전후 ±3 문장

사용처:

- GET /api/patient/sentences/{file} — 환자 페이지 근거 문장
- DB에서 분석 결과 재구성

3. doctor_sentence_view (221행)

의사 대시보드 문장 목록 — 의사가 리뷰하고 필요 시 수정(rewrite)할 문장을 중복 없이 저장하는 테이블.

sentence_prediction에서 같은 문장이 여러 도메인에 나타날 수 있지만, 여기서는 각 문장이 대표 도메인 1개와 함께 1번만 나타남.

이 테이블이 221행인 이유(sentence_prediction의 250행보다 적음)는 중복 제거: 같은 문장이 cp와 le 도메인 모두에서 Top-10에 선택되면 sentence_prediction에는 2행이지만 doctor_sentence_view에는 1행만 존재합니다. 의사는 각 문장을 정확히 1번만 봅니다. 의사가 문장을 수정할 때 이 테이블의 (file, i, i2) 가 대상 문장을 식별합니다.

컬럼	타입	실제 값	설명
----	----	------	----

file	VARCHAR PK	Input_Keystrokes REC 001 (SID 10).xlsx	환자 파일
i	INT PK	67	발화 번호
i2	INT PK	3	발화 내 문장 번호
speaker	VARCHAR	Interviewer:	화자
sentence	TEXT	"so i'm going to take that 12 percent..."	문장 텍스트
score	FLOAT	None	품질 점수 (나중에 채울 수 있음)
class	VARCHAR	cancer_prognosis	대표 도메인
time	TIMESTAMP	05:26:30	생성 시각

사용처:

- GET /api/doctor/sentences/{file}/{speaker} — 의사 페이지 문장 목록
- GET /api/doctor/files — 처리된 환자 파일 목록 (DISTINCT file)
- PUT /api/doctor/rewrites — 수정 대상 문장 식별 (file, i, i2)
- GET /api/patient/sentences/{file} — 환자 페이지 근거 문장

4. doctor_rewrite_log (0행)

의사 수정 이력 — 의사가 대시보드에서 문장을 수정할 때마다 원본과 수정본을 타임스탬프와 함께 기록하는 이력 테이블.

현재 0행인 이유는 아직 의사가 문장을 수정한 적이 없기 때문입니다. 의사가 "so i'm going to take that 12 percent..."를 "Your cancer risk is about 6% with treatment"로 수정하면, 원본과 수정본이 함께 INSERT됩니다. 같은 문장을 여러 번 수정하면 여러 행이 쌓여서 전체 수정 이력을 추적할 수 있습니다.

컬럼	타입	예시 값	설명
file	VARCHAR PK,FK	Input_Keystrokes REC 001 (SID 10).xlsx	대상 파일
i	INT PK,FK	67	대상 발화 번호
i2	INT PK,FK	3	대상 문장 번호
time	TIMESTAMP PK	2026-04-17T14:30:00Z	수정 시각 (같은 문장 여러 번 수정 가능)
speaker	VARCHAR	Interviewer:	화자
original_sentence	TEXT	"so i'm going to take..."	원본 문장
revised_sentence	TEXT	"Your cancer risk is about 6%..."	의사가 수정한 문장
score	FLOAT	4.0	수정 후 품질 점수
class	VARCHAR	cancer_prognosis	도메인

사용처:

- GET /api/doctor/rewrites — 의사 페이지 수정 이력

- GET /api/doctor/rewrites/{file}/{i}/{i2}/history — 문장별 수정 이력

5. patient_summary (5행)

환자 요약 부모 레코드 — "이 환자에 대한 요약이 존재한다"를 나타내는 테이블. **patient_summary_domain**(환자당 5개 도메인 행)의 FK 부모.

환자당 1행만 존재합니다. 실제 도메인별 데이터(summary_text, patient_scoring)는 자식 테이블 **patient_summary_domain**에 있습니다. 이 테이블이 필요한 이유는 DB 설계상 1:N 관계에서 "1" 쪽 테이블이 있어야 FK 무결성이 보장되기 때문입니다. 여기서 환자를 삭제하면 5개 도메인 행도 CASCADE 삭제됩니다.

컬럼	타입	실제 값	설명
file	VARCHAR PK	Input_Keystrokes REC 001 (SID 10).xlsx	환자 파일
speaker	VARCHAR PK	Patient_Input_Keystrokes REC 001 (SID 10)	환자 식별자
entire_summary	TEXT	None	전체 방문 요약 (현재 미사용)

사용처:

- GET /api/patient/summaries/{file}/{speaker} — patient_summary_domain과 JOIN
- patient_summary_domain의 FK 부모 (CASCADE 삭제)

6. patient_summary_domain (25행 = 5환자 × 5도메인)

환자 도메인별 피드백 — 환자가 대시보드에서 도메인별로 별점과 텍스트 응답을 입력하면 저장되는 테이블. 파이프라인이 빈 행 (NULL 값)을 생성하고, 환자가 재진 시 채움.

환자 설문지와 같음: 파이프라인이 5개 도메인(cp, le, ed, inc, ius)의 빈 카드를 생성하고, 환자가 "암 예후" 별점을 클릭하면 **patient_scoring**이 NULL에서 평점 값으로 UPDATE됩니다. 참고: **patient_response** (자유 텍스트)는 API는 있지만 UI가 아직 구현되지 않았습니다.

컬럼	타입	실제 값 (SID_10, cp)	설명
file	VARCHAR PK,FK	Input_Keystrokes REC 001 (SID 10).xlsx	환자 파일
speaker	VARCHAR PK,FK	Patient_...SID 10	환자 식별자
domain	VARCHAR PK	cancer_prognosis	도메인
display_order	INT	1	UI 표시 순서 (1=cp, 2=inc, 3=ed, 4=ius, 5=le)
summary_text	TEXT	""	도메인 요약 텍스트 (현재 비어있음)

patient_scoring	INT	None	환자 별점 0-10 (환자 입력 전 NULL)
patient_response	TEXT	None	자유 텍스트 피드백 (API 있지만 UI 미구현)

사용처:

- GET /api/patient/summaries/{file}/{speaker} — 환자 페이지 도메인별 요약 카드
- PUT /api/patient/scoring — 환자 별점 입력 (PatientInitialVisitReportV35.tsx)
- PUT /api/patient/responses — 환자 텍스트 응답 (API만 존재, UI 미구현)

7. survey_submission_log (0행)

환자 설문 응답 — 환자가 재진 페이지에서 **SDM, DCS, Risk Perception, Satisfaction** 설문을 제출하면 응답 전체가 **JSON**으로 저장되고, **REDCap** 연구 데이터베이스와 동기화 상태를 추적하는 테이블.

현재 0행인 이유는 아직 환자가 설문을 제출한 적이 없기 때문입니다. 환자가 DCS 16문항에 응답하면 전체 응답이 **answers**에 JSON으로 저장됩니다. 저장 직후 백그라운드에서 REDCap API를 호출하여 동기화합니다. 같은 환자가 같은 설문을 다시 제출하면 UPDATE가 아닌 INSERT(새 행)로 시점별 응답을 모두 보존합니다.

컬럼	타입	예시 값	설명
id	SERIAL PK	1	제출 고유 ID
file	VARCHAR FK	Input_Keystrokes REC001 (SID 14).xlsx	환자 파일
speaker	VARCHAR FK	Patient_...SID 14	환자 식별자
survey_type	VARCHAR	dcs	설문 종류 (sdm/dcs/risk_perception/satisfaction)
answers	JSONB	{q1: 2, q2: 3, ..., q16: 4}	설문 응답 전체
extra_data	JSONB	{browser: Chrome, session: abc}	메타데이터
submitted_at	TIMESTAMP	2026-04-17T16:05:30Z	제출 시각
redcap_synced	BOOLEAN	true	REDCap 동기화 성공 여부
redcap_record_id	VARCHAR	REC-2026-0014	REDCap 레코드 ID (성공 시)
redcap_error	TEXT	None	에러 메시지 (실패 시)

사용처:

- POST /api/surveys/submit — 환자 재진 페이지에서 설문 제출
- GET /api/surveys/by-speaker/{speaker} — 페이지 새로고침 시 이전 응답 복원
- 백그라운드 REDCap 동기화 워커

8. llm_domain_scoring_and_summary (33행)

GPT-4o AI 파이프라인 결과 — "의사가 환자에게 위험을 얼마나 구체적으로 전달했는가"에 대한 도메인별 평가 결과를 저장하는 테이블. 0-5 점수, 추출된 수치 추정값, 환자 친화적 변환 텍스트 포함.

NLP(sentence_prediction)가 "어떤 문장이 이 도메인과 관련있는가?"를 판단했다면, 이 테이블은 "그 문장에서 의사가 얼마나 구체적으로 위험을 전달했는가?"를 저장합니다. 예: "암 위험이 있습니다" (모호) = ai_score 1, "치료 없이 24%, 치료 시 6%로 감소" (매우 구체적) = ai_score 4. 33행(5x5=25보다 많음)인 이유는 일부 도메인에서 여러 치료법 비교가 별도 행으로 저장되기 때문입니다.

컬럼	타입	실제 값 (SID_10, cp)	설명
id	SERIAL PK	1	결과 고유 ID
analysis_id	INT FK	1	→ transcript_analysis_log.id
patient_id	VARCHAR	SID_10	환자 식별자
domain	VARCHAR	cp	도메인
ai_score	INT	4	GPT-4o 점수 0-5 → 의사 페이지
score_explanation	TEXT	"Does the text mention cancer mortality?... score is 4."	GPT-4o 추론 과정 (ai_pipeline/scoring.py에서 생성)
extracted_estimate	TEXT	"24, 25 percent—down to six percent"	추출된 위험 수치
treatment	VARCHAR	None	관련 치료법 (ed/inc/ius만 해당)
source_sentence	TEXT	"so i'm going to take that 12 percent..."	AI가 선택한 원본 문장
source_context	TEXT	"it removes the testosterone...<main>...</main>..."	주변 문맥
reformat_sentence	TEXT	"Your doctor noted that your risk of dying of cancer is 24–25%..."	환자 친화적 텍스트 → 환자 페이지
source_filename	VARCHAR	Input_Keystrokes REC 001 (SID 10).xlsx	원본 파일
created_at	TIMESTAMP	05:29:28	생성 시각

사용처:

- GET /api/doctor/scores/average — 의사 페이지 상담 품질 점수
- GET /api/doctor/scores/trajectory — 의사 페이지 시간별 점수 추이
- GET /api/patient/ai-summary/{file} — 환자 페이지 AI 요약 카드
- POST /api/doctor/ai-rewrite — source_sentence + source_context로 AI 수정 제안

9. user_interaction_log (108행)

사용자 행동 추적 — 대시보드에서의 모든 사용자 행동(클릭, 스크롤, 마우스 이동, 체류 시간)을 실시간 기록하여, 환자와 의사가 상담 정보와 어떻게 상호작용하는지 연구 분석용으로 저장하는 테이블.

예를 들어 환자가 "암 예후" 카드 근처에 38초간 마우스를 두면, `event_type=cursor_proximity_leave` 와 `hoverDuration: 38007071ms` 이벤트가 기록됩니다. 이 데이터로 "환자가 어떤 도메인 정보에 가장 오래 머물렀는가?", "의사가 어떤 문장을 가장 많이 클릭했는가?" 같은 연구 질문에 답할 수 있습니다.

컬럼	타입	실제 값	설명
id	SERIAL PK	1	이벤트 고유 ID
session_id	VARCHAR	session_1776435900666_syzu2gq6lrk	브라우저 세션 ID
role	VARCHAR	patient	사용자 역할 (patient/physician)
visit_type	VARCHAR	first	방문 유형 (first/followup)
file	VARCHAR	Input_TurboScribe SID 33.csv	조회 중인 환자
speaker	VARCHAR	Patient_Input_TurboScribe SID 33	환자 식별자
event_type	VARCHAR	cursor_proximity_leave	이벤트 종류
element_id	VARCHAR	/	대상 UI 요소
event_data	JSONB	{cursorX: 0, hoverDuration: 38007071, ...}	이벤트 상세 데이터
device_type	VARCHAR	desktop	기기 타입
client_timestamp	TIMESTAMP	14:25:00	클라이언트 시각
created_at	TIMESTAMP	14:25:00	서버 저장 시각

사용처:

- POST `/api/tracking/events` — 프론트엔드에서 이벤트 배치 전송
- GET `/api/tracking/stats` — 관리자 추적 대시보드
- GET `/api/tracking/analytics` — 행동 분석

10. session_recording (2행)

세션 리플레이 데이터 — `rrweb`으로 녹화한 사용자 세션을 바이너리 청크로 저장하여, 사용자가 대시보드를 어떻게 탐색했는지 영상처럼 재생할 수 있게 하는 테이블. PHI(보호건강정보)가 마스킹되어 환자 이름과 문장 텍스트는 녹화에서 `"`로 치환됨.*

`user_interaction_log` 가 개별 이벤트를 기록한다면, 이 테이블은 전체 세션을 시각적으로 재생하는 데 필요한 raw 데이터를 저장합니다. 30초마다 또는 500개 이벤트마다 청크 단위로 서버에 전송됩니다. 관리자 추적 대시보드에서 이 녹화를 재생하여 사용자 행동을 관찰할 수 있습니다.

컬럼	타입	실제 값	설명
----	----	------	----

id	SERIAL PK	1	녹화 고유 ID
session_id	VARCHAR	rec_1776397890521_hp3r5x	녹화 세션 ID
chunk_index	INT	0	청크 번호 (30초 또는 500이벤트마다 분할)
file	VARCHAR	Input_TurboScribe SID 33.csv	조회 중이던 환자
visit_type	VARCHAR	first	방문 유형
recording_data	BYTEA	(87 bytes)	rrweb 이벤트 데이터 (PHI 마스킹 적용)
event_count	INT	2	이 청크의 이벤트 수
created_at	TIMESTAMP	14:25:30	서버 저장 시각

사용처:

- `POST /api/tracking/recordings` — 프론트엔드에서 청크 업로드 (sessionRecorder.ts)
- `GET /api/tracking/recordings` — 관리자 대시보드 녹화 목록
- `GET /api/tracking/recordings/{sessionId}` — 세션 리플레이 재생